

# GridPulse

## AI-Powered Renewable Generation Forecasting Platform

Team [ add team name ] Users grid operators · utilities · plant owners · energy traders Horizon 24–72 h, hourly Status working prototype

Every utility already receives tomorrow's weather forecast. **Nothing turns it into tomorrow's power, hour by hour, per site — then flags the hours that breach that site's limits and states the action.** GridPulse is that missing conversion layer.

### 1 Problem Statement

Solar and wind output swings constantly with weather, time of day and season. Grid operators must commit capacity, schedule backup and respect grid connection limits hours ahead — every one of those decisions made on a forecast. When the forecast is weak, the cost lands in three places:

- **Wasted clean energy.** Surplus that could have been stored or sold is curtailed, because nobody saw it early enough to place it.
- **Unnecessary fossil backup.** Reserve plant is held spinning against uncertainty rather than against a known shortfall.
- **Unsize financial exposure.** A plant that sold power day-ahead cannot tell, in advance, which hours it will fail to deliver.

The gap is not data — it is the conversion. Weather forecasts are public and free. What no one hands an operator is: *this site will make 100.6 MW at 07:00, which is 106% of your substation limit, so curtail 5.6 MW.*

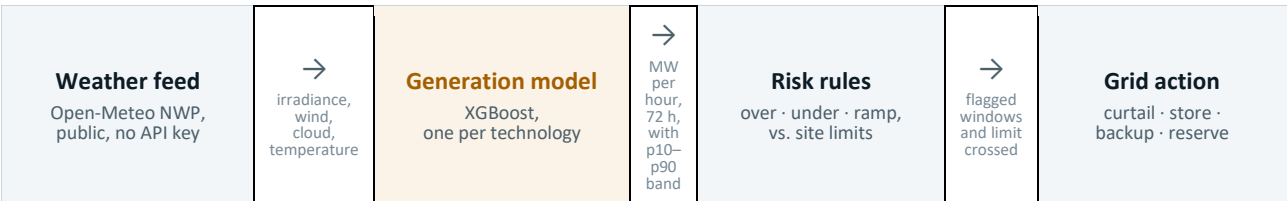
### 2 Proposed Solution

GridPulse is an intelligence platform that ingests weather forecasts, historical generation records and site-level parameters, and returns a 24–72 hour generation forecast, flagged risk windows, and an explained grid action for each one.

- **Forecast** — hourly predicted output per site with an 80% confidence band, derived from the same numerical weather prediction a utility already receives.
- **Flag** — over-generation, under-generation and ramp windows, measured against that site's real operating limits rather than a generic percentage of nameplate.
- **Recommend** — curtailment, storage charge or dispatch, backup activation or spinning reserve, each carrying a reason sentence assembled from the numbers behind it.

**Exactly one component is learned.** The weather-to-power conversion is machine learning, because that relationship is non-linear and worth fitting. Everything after it is deterministic arithmetic an operator can audit line by line — because nobody curtails 100 MW on the grounds that a model said so.

### 3 Approach



The pipeline. One learned component converts weather into power; every decision after it is a comparison between two numbers the dashboard also puts on screen.

**Forecast weather, then convert it** — rather than predicting power directly. This is how renewable forecasting genuinely works in industry, and it means a site with no operating history is forecastable from its coordinates alone.

**Build the training history from physics** — no utility hands out SCADA data during a hackathon, and random numbers teach a model nothing. We replay 15 months of real ERA5 reanalysis weather per site through first-order generation physics — a PV performance-ratio model (PR 0.78,  $-0.4\%/^{\circ}\text{C}$  above  $25^{\circ}\text{C}$ ) and a four-region turbine power curve (cut-in 3.5, rated 12.5, cut-out 25 m/s) — plus 6% stochastic noise. That yields 65,808 weather-correlated site-hours carrying the diurnal cycle, seasonal swing, cloud events and storm shutdowns a model needs to learn. Swapping in real SCADA is a file change, not a redesign.

**Predict capacity factor, not kilowatts** — so one model per technology serves a 10 MW rooftop and a 240 MW offshore array alike, error reads directly as a percentage of capacity, and adding a new site requires no retraining.

**Run the horizon recursively** — lagged generation is real signal and we do not know next Tuesday's output, so each predicted hour feeds the next as its lag. Confidence bands come from held-out residuals bucketed by predicted output, widened with lead time.

**Prove it honestly** — a chronological holdout — never shuffled, which would let the model see 13:00 and 15:00 while predicting 14:00 — scored against the persistence baseline a control room actually falls back on, plus a rolling day-ahead backtest in which the model is rewound each morning and never shown the answers.

### Validation results

| METRIC (HELD-OUT DATA, NEVER SEEN IN TRAINING)                      | SOLAR        | WIND         |
|---------------------------------------------------------------------|--------------|--------------|
| Mean absolute error, % of nameplate capacity                        | <b>0.97%</b> | <b>1.43%</b> |
| Root mean squared error, % of capacity                              | 1.92%        | 2.28%        |
| $R^2$                                                               | 0.9931       | 0.9932       |
| Persistence baseline MAE (what a control room does without a model) | 5.24%        | 5.94%        |
| Improvement over that baseline                                      | <b>81.4%</b> | <b>75.9%</b> |
| Held-out hours scored                                               | 6,579        | 6,579        |

**How to read these numbers.**  $R^2$  is 0.993 for both models because the training targets come from a physics model plus noise, so the mapping being learned genuinely exists in the data. Against real SCADA, day-ahead solar forecasts typically land nearer 4–8% MAE — real plants also have outages, soiling and curtailment no weather feed predicts. Read these as evidence the model has correctly learned the weather-to-power relationship. The figure that transfers is the persistence comparison.

## 4 Key Features

### Portfolio view

- World map with a pin per site, colour-coded by current alert status.
- KPIs: capacity online, generating now, forecast for the next 24 h, sites needing action, and how much of the at-risk curtailment on-site storage can actually absorb.
- A cross-portfolio action queue ranked by urgency, so one screen covers the whole estate.

### Site detail

- 72-hour forecast chart with a shaded confidence band, recorded output stitched to the forecast across a NOW marker, and each flagged risk window shaded on the chart itself rather than listed separately.
- Operating limits drawn as reference lines — export limit, over-generation trigger, under-generation floor — so a breach is visible, not inferred.
- A “yesterday's forecast vs actual” panel scored in MAE, RMSE and  $R^2$ , plus the weather driving the prediction: irradiance, cloud cover, hub-height wind and temperature.

### Flagging that reflects real constraints

- **Over-generation** — above 85% of the site's grid export limit. A plant that can make 180 MW behind a 95 MW substation is over-generating at 81 MW, whatever headroom the panels still have. Connection limits cause most real curtailment.
- **Under-generation** — below the greater of 15% of capacity and the site's firm delivery schedule. A plant that sold 45 MW day-ahead is short at 30 MW even though 30 MW is healthy output. That is the number a trader is exposed on.

- **Ramp events** — swings past 25% of capacity in an hour, merged into single events so a front crossing a wind farm is one thing to schedule against, not four separate alerts.

### Recommendations that explain themselves

- Storage handling is quantitative, not a lookup: the engine sizes the surplus against real battery headroom, tracks state of charge forward through the horizon, and where the battery covers only part of the surplus recommends both actions — charge what fits, curtail the rest.
- Every sentence is assembled from live figures, so the wording moves with the forecast. Nothing in the interface is hardcoded demo text.

“Forecast peaks at 100.6 MW across 4h (06:00–09:00 UTC on 13 Sep), 106% of the 95.0 MW substation export limit. The 60 MWh battery is already at 100% and has no headroom left, leaving 9.2 MWh with nowhere to go — issue a curtailment instruction capped at 95.0 MW export, trimming up to 5.6 MW at the peak hour.”

VERBATIM FROM `GET /api/sites/{id}/recommendations`

## 5 Technology and Implementation Plan

|                         |                                                                                                                                                                                                                              |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Backend</b>          | Python · FastAPI · SQLAlchemy + SQLite · pandas · APScheduler for scheduled forecast refresh                                                                                                                                 |
| <b>Machine learning</b> | XGBoost gradient-boosted regressors, one per technology · scikit-learn for metrics · 20 engineered features per hour, including cyclical time encoding, NOAA solar geometry, clear-sky index, wind shear and generation lags |
| <b>Frontend</b>         | Next.js 15 · TypeScript · Tailwind CSS · Recharts · react-leaflet · Framer Motion                                                                                                                                            |
| <b>Data sources</b>     | Open-Meteo forecast API and ERA5 reanalysis archive API — free, keyless, no signup, no rate-limit management                                                                                                                 |
| <b>Serving</b>          | 9 REST endpoints; each site's forecast computed once and cached, refreshed on a schedule and warmed at startup, so the dashboard answers in 10–40 ms                                                                         |

No paid APIs and no service requiring a credit card or an approval wait. The whole system runs from a clean clone with documented install commands and an empty .env file.

### Build sequence

| STAGE              | DELIVERABLE                                                                                                                                                           | STATUS          |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| <b>Data layer</b>  | Open-Meteo client, PV and turbine physics, 15-month historical dataset builder                                                                                        | <b>Complete</b> |
| <b>ML pipeline</b> | Feature engineering shared by training and serving, chronological training, metrics written to file                                                                   | <b>Complete</b> |
| <b>API + rules</b> | Sites, weather, forecast, backtest, alerts, recommendations, portfolio summary; flagging and recommendation engines                                                   | <b>Complete</b> |
| <b>Dashboard</b>   | Portfolio map and KPIs, site detail view, backtest panel, model report                                                                                                | <b>Complete</b> |
| <b>Next</b>        | Quantile regression for calibrated p10/p50/p90 bands; multi-model NWP ensemble, using cross-model spread as its own uncertainty signal                                | Planned         |
| <b>Then</b>        | SCADA ingestion replacing the synthetic history; Postgres + TimescaleDB at fleet scale; price-aware curtailment economics; scheduled retraining with drift monitoring | Planned         |
| <b>Later</b>       | Write-back into the dispatch system behind an operator sign-off step, so a recommendation becomes an instruction                                                      | Planned         |

**Prototype already running.** Six real sites totalling 892 MW across four continents — three solar, three wind — forecasting on live weather, with the flagging and recommendation engines operating end to end. The architecture above is not drawn on paper; each remaining row is a swap behind an interface that already exists.